SideQuest: from the zip file to Google Play (baby-step guide)

You have done nothing yet. That is fine. This guide assumes **nothing is installed and no accounts exist**. Follow it top to bottom. Tick the boxes (☐ → ☑) as you go. Do not skip steps.

Version of the code this guide is written for: `sidequest-v11.zip`. Written 7 November 2026.

PART 0. Read this first (5 minutes)

0.1 What you are going to build

Piece	What it is	Where it lives
The **app**	What people install on their Android phone	Built by Expo (cloud), installed from Google Play
The **database + sign-in + photo storage**	All the data and who can see what	**Supabase** (website)
The **API + worker**	A small server that sends push notifications, builds data exports, deletes accounts, serves the moderator page and the legal pages	**Railway** (website)
The **moderator page**	Where you approve photos and handle reports	`https://YOUR-API/admin` (served by the API)
The **legal pages**	Privacy policy, terms, "delete my account" page. Google Play **requires** these	`https://YOUR-API/privacy`, `/terms`, `/delete-account`

0.2 How long and how much (be realistic)

Phase	Time
Accounts + installing tools (Parts 1–3)	1–2 hours
Supabase + Railway set-up (Parts 4–6)	2–4 hours
First build on your own phone and testing (Parts 7–8)	1 day
Google Play developer account approval	A few days (identity checks)
Mandatory closed test for new personal Play accounts: 12 people, 14 days in a row	**14+ days**
Google's review of your production request	Up to about 7 days (can be longer for dating apps)

Plan on about 4–5 weeks from today to "anyone can download it". The 14-day test cannot be skipped if your Google developer account is a *personal* one created after 13 November 2023 (Google's rule, confirmed on Google's own help page in November 2026). Organization accounts are exempt but need a D-U-N-S number (can take weeks). See Part 11.

Cost	Roughly
Google Play developer account	**US$25 once**
Supabase	Free plan to start. **Before real users, plan for the paid plan (about US$25/month)**: free projects pause after inactivity and have no backups. Check supabase.com/pricing
Railway	A few US$ per month (check railway.com/pricing; you need two small services)
Expo (builds)	Free plan has a limited number of builds per month (check expo.dev/pricing)
Firebase (push notifications)	Free
Email sender for sign-in codes (Brevo/Resend etc.)	Free tier is enough to start
A web address of your own	Optional. Railway gives you a free one. A domain is about US$10–15/year

Prices change. Check each site before you pay.

0.3 Honest status. Please read.

- **Nothing in this project has ever been run on a real phone, on a real Supabase project, on Railway, or on Google Play.** It was written and tested in a sandbox: the logic tests pass, the database was run on a normal Postgres with stand-ins for Supabase, the screens were rendered in a fake browser, and the app builds its Android project correctly (target API level 36, which Google requires). That is strong evidence, **not proof**.
- **So you are the first real user.** Expect to hit a few errors. That is normal. Part 0.4 tells you exactly what to send me when it happens, and I will fix it with you.
- The code covers **Android first**. iPhone/App Store is in Part 13 and is *not set up or tested*.

0.4 When something goes wrong (how to ask me for help)

Copy and send me **all** of these:

1. The **step number** from this guide (for example "Step 5.7").
2. What you **did** (the exact command or the button you pressed).
3. The **exact error text** (copy and paste it; a screenshot is fine too).
4. **Blank out secrets** first: database passwords, anything that looks like `eyJhbGciOi...` (keys), and `sb_secret_...`. Never paste a key into a chat.

0.5 Words you will see

- **Terminal** = the window where you type commands. **Repository (repo)** = the folder of code stored on GitHub. **Build** = turning the code into an installable Android file. **APK** = an Android file you can install directly. **AAB** = the file you upload to Google Play. **Environment variable ("variable")** = a named setting, like `SUPABASE_URL=...`. **Migration** = a numbered SQL file that sets up the database.

0.6 Your secrets sheet

Open a note (a password manager is best) and create this list now. You will fill it in as you go. **Never share these publicly.** The two marked 🔴 must **never** go into the app or into GitHub.

Name	Value (fill in later)	Where it is used
Supabase project URL		app, API, worker
Supabase **anon** key (starts `eyJ…`)		app, API, worker (public by design)
🔴 Supabase **service_role** key (starts `eyJ…`)		API, worker only
🔴 Supabase database connection string (with password)		API, worker only
Supabase JWT secret (only if needed, Step 4.6)		API, worker
Railway API address (`https://….up.railway.app`)		app
🔴 WEBHOOK_SECRET (you invent it: 20+ random characters)		API, Supabase webhook
🔴 Firebase service account JSON (Part 8)		API, worker
Expo account login		building
Google Play account login (turn on 2-step verification!)		publishing

---

PART 1. Create your accounts (30–45 minutes)

Use one email address that **only you** control and check daily. Turn on **2-step verification** wherever offered.

☐ **Step 1.1. GitHub** → github.com → *Sign up*. (Holds your code; Railway and Supabase sign-in use it.) ☐ **Step 1.2. Supabase** → supabase.com → *Start your project* → *Continue with GitHub*. ☐ **Step 1.3. Railway** → railway.com → *Login* → *Login with GitHub*. ☐ **Step 1.4. Expo** → expo.dev → *Sign up*. Remember your **username**. ☐ **Step 1.5. Firebase** → console.firebase.google.com → sign in with a Google account. (Only used for push notifications.) ☐ **Step 1.6. An email-sending service** for sign-in codes (Supabase's built-in email is only for testing; see Step 4.9). Easiest for beginners: **Brevo** (brevo.com), free plan. Create the account and verify your own email address as a sender. ☐ **Step 1.7. Google Play Console: do this now, not later**, because identity checks take days. Go to **Part 11, Steps 11.1–11.3**, do just those, then come back here. ☐ **Step 1.8.** An **Android phone** (Android 8 or newer) and a USB cable (optional). You will also need **12 other people with Android phones** later (Part 12). ☐ **Step 1.9.** Make a **separate Gmail address for Google's app reviewers** (for example `sidequest.review@gmail.com`) with a password you will share *only* in the Play Console review notes. Reviewers cannot receive emails sent to you, so they will sign in to the app using this mailbox (Step 11.8).

PART 2. Install the tools on your computer (20–40 minutes)

You need **Node.js**, **Git**, and a text editor. Windows, Mac and Linux all work.

☐ **Step 2.1. Open a Terminal.**

- **Windows:** press the Windows key, type `PowerShell`, press Enter.
- **Mac:** press Cmd+Space, type `Terminal`, press Enter.
- **Linux:** you know.

☐ **Step 2.2. Install Node.js.** Go to **nodejs.org**, download the **LTS** version (current LTS is fine; version **22.18 or newer, or 24.x**), run the installer, accept the defaults. **Close the terminal and open a new one.** Then type:

```
node -v
```

✅ You should see a number like `v24.x.x` or `v22.x.x`. ❌ If you see "not recognized/command not found", reinstall and reopen the terminal.

☐ **Step 2.3. Install Git.** git-scm.com → download → install with defaults. New terminal, then:

```
git --version
```

✅ You see `git version 2.x`.

☐ **Step 2.4. Install a text editor:** code.visualstudio.com (VS Code, free). You will use it to open and edit a few files.

☐ **Step 2.5. Turn on pnpm** (the tool that installs the code's dependencies):

```
corepack enable
pnpm -v
```

✅ You see `9.15.0` (it may say "Corepack is about to download pnpm", answer **Y**). ❌ If `corepack` is not found, run `npm install -g pnpm@9.15.0` instead.

☐ **Step 2.6. Install the Expo build tool (EAS):**

```
npm install -g eas-cli
eas --version
```

✅ You see a version number. (On Mac/Linux, if you get a permission error, put `sudo` in front of the first command.)

PART 3. Get the code and check your computer can run it (20 minutes)

☐ **Step 3.1.** Download `sidequest-v11.zip` from the chat to your computer (put it in your Documents folder).

☐ **Step 3.2. Unzip it.** Windows: right-click → *Extract All*. Mac: double-click. You now have a

folder named `sidequest`. Everything below happens inside it.

☐ **Step 3.3.** In your terminal, go into the folder (change the path to where you put it):

```
cd Documents/sidequest
```

Windows PowerShell example: `cd $HOME\Documents\sidequest`. To check you are in the right place, type `dir` (Windows) or `ls` (Mac/Linux). ✅ You see `apps`, `docs`, `packages`, `supabase`, `package.json`.

☐ **Step 3.4. Install everything the code needs:**

```
pnpm install
```

✅ After a minute or two it ends with `Done in …s`. (Warnings are okay; red `ERR` lines are not.)

☐ **Step 3.5. Check your computer is working with this code.** Run:

```
pnpm test
```

This takes a few minutes. ✅ It ends **without** the words `ERR_PNPM`, and you see three blocks that each show `# fail 0`. (One block says `# skipped 8`: those need a database and are skipped on purpose.) Then:

```
pnpm typecheck
pnpm check:sql
```

✅ `SQL static checks passed` at the end.

☐ **Step 3.6. Open the project in your editor:** `code .` (or open VS Code → *Open Folder* → pick `sidequest`).

> If any of 3.4–3.6 fails, stop and send it to me (Part 0.4). Do not continue with a broken base.

PART 4. Supabase: database, sign-in, photo storage (1–2 hours)

Create the project

☐ **Step 4.1.** supabase.com/dashboard → *New project*. Name: `sidequest`. **Database password:** click *Generate a password*, **copy it to your secrets sheet.** Region: pick the one **closest to most of your users.** Plan: Free for now. *Create new project*. Wait 2–3 minutes until it says the project is ready.

☐ **Step 4.2. Collect the keys.** Open **Project Settings** (gear icon) → **API Keys.** Supabase now shows two sets: new "publishable/secret" keys and a **"Legacy API keys"** tab. **Use the Legacy tab:** this code was written for the older key format that starts with `eyJ`.

- Copy **Project URL** (looks like `https://abcdxyz.supabase.co`) → secrets sheet.
- Copy the `anon` key → secrets sheet.

- Copy the `service_role` key → secrets sheet 🔴 (reveal it with the eye icon).

> If the Legacy keys are switched off for your project, tell me. Do not guess.

☐ **Step 4.3. Collect the database connection string.** Click **Connect** at the top of the dashboard → find **Session pooler** → copy the **URI**. It looks like `postgresql://postgres.xxxx:[YOUR-PASSWORD]@aws-0-....pooler.supabase.com:5432/postgres`. Replace `[YOUR-PASSWORD]` (including the square brackets) with the database password from Step 4.1. Save to the secrets sheet 🔴.

Create the database tables (13 SQL files, in order)

☐ **Step 4.4.** In Supabase click **SQL Editor** (left side) → **New query**.

For **each** file below, in this order: open it in VS Code (it is in `sidequest/supabase/migrations/`), press **Ctrl+A** (select all) then **Ctrl+C** (copy), paste into the Supabase SQL editor (replace what is there), and click **Run**. ✅ Each one ends with **"Success. No rows returned"**. ❌ If any file shows an **error**, **stop** and send me the error and the file name. Do not run the next ones.

☐	File
☐	`0001_core_schema.sql`
☐	`0002_social_features.sql`
☐	`0003_rls_and_rpc.sql`
☐	`0004_api_support.sql`
☐	`0005_onboarding_and_hardening.sql`
☐	`0006_discovery_and_likes.sql`
☐	`0007_chat.sql`
☐	`0008_open_calls_activities.sql`
☐	`0009_social_features_rpc.sql`
☐	`0010_safety_privacy.sql`
☐	`0011_worker_support.sql`
☐	`0012_moderation_tools.sql`
☐	`0013_app_support.sql`

☐ **Step 4.5. Run the safety self-test.** Paste the whole of `supabase/tests/rls_smoke.sql` into a new query and Run. ✅ The result table shows `ALL RLS SMOKE TESTS PASSED`. (It changes nothing permanently.) ❌ Any other result: send it to me. **Never run** `local_stubs.sql` (it is for a different purpose and would break things).

☐ **Step 4.6. Check how Supabase signs logins.** Project Settings → **JWT Keys** (or "JWT Settings"). If it says your current signing key is an **asymmetric key (ECC/RSA)**, you need nothing. If it shows only a **legacy JWT secret (HS256)**, copy that secret into your secrets sheet as "JWT secret"; you will add it to Railway in Part 5.

☐ **Step 4.7. Check the storage buckets exist.** Left side **Storage**. ✅ You see `profile-photos`, `chat-media`, `voice-intros` and `exports`, all marked private.

Sign-in emails

The app signs people in with a **6-digit code sent by email** (no passwords).

☐ **Step 4.8. Turn on email sign-in and put the code in the email.**

- **Authentication → Sign In / Providers → Email:** make sure **Email** is **enabled** and **Confirm email** is **on**.
- **Authentication → Email Templates.** There are two templates that matter: **Confirm signup** (sent to brand-new people) and **Magic Link** (sent to people who already have an account). Edit **both** so the body contains the code. Replace the body with:

```
<h2>Your SideQuest code</h2>
<p>Enter this code in the app: <strong>{{ .Token }}</strong></p>
<p>If you did not ask for this, ignore this email.</p>
```

Click **Save** on each.

☐ **Step 4.9. Set up a real email sender (important).** Supabase's built-in email sender is **only for testing**: it has a very small limit and, on a project without your own sender, it may send only to people on your Supabase team. Real users would never get their codes.

- In **Brevo**: *SMTP & API* → copy the **SMTP server** (usually `smtp-relay.brevo.com`), **port** `587`, your **login**, and create an **SMTP key** (this is the password). Make sure the sender address you will use is verified in Brevo.
- In **Supabase**: **Authentication → Emails → SMTP Settings** (name may differ slightly) → turn on **Custom SMTP** → enter the host, port 587, username, password (the SMTP key), **Sender email** (your verified address) and **Sender name** `SideQuest` → Save.
- Test it later in Part 7 by signing in with your own email.

☐ **Step 4.10.** *Authentication → Rate Limits*: leave the defaults for now.

PART 5. Put the code on GitHub, then run the server on Railway (1–2 hours)

Upload your code to GitHub

☐ **Step 5.1.** github.com → **New repository** → name `sidequest` → **Private** → do **not** tick "Add a README" → *Create repository*. GitHub shows a page with commands; keep it open.

☐ **Step 5.2.** In your terminal, inside the `sidequest` folder, run these one at a time (replace `YOUR-USERNAME`):

```
git init
git add .
git commit -m "first upload"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/sidequest.git
git push -u origin main
```

(Git may ask you to sign in to GitHub in a browser window. Do it.) ✅ Refresh the GitHub page: you see your folders. The file `.gitignore` already keeps your `.env` files and keystores **out** of GitHub. After `pnpm install` created a file named **`pnpm-lock.yaml`**, it is included, which is what you want.

> Never type a key into any file that is committed. Real keys go only into Railway/Expo settings and your private `.env` (which is ignored).

Railway: the API service

☐ **Step 5.3.** railway.com/dashboard → **New Project** → **Deploy from GitHub repo** → choose `sidequest` (allow Railway to access it if asked). Railway starts a first deploy that **will fail**. That is normal; you have not given it settings yet.

☐ **Step 5.4.** Click the new service → **Settings**:

- **Root Directory:** `/`
- **Config as code → path:** `/apps/api/railway.json` ← *(not just `/railway.json`)*
- Rename the service to `api`.

☐ **Step 5.5. Variables.** Click the **Variables** tab → *Raw Editor* → paste the following, then **replace every** **`PASTE_...`** using your secrets sheet. One variable per line, no quotes:

```
DATABASE_URL=PASTE_SESSION_POOLER_STRING
SUPABASE_URL=PASTE_PROJECT_URL
SUPABASE_ANON_KEY=PASTE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY=PASTE_SERVICE_ROLE_KEY
WEBHOOK_SECRET=PASTE_20_PLUS_RANDOM_CHARACTERS
LEGAL_NAME=PASTE_YOUR_FULL_NAME_OR_COMPANY_NAME
CONTACT_EMAIL=PASTE_A_SUPPORT_EMAIL_YOU_CHECK_DAILY
CHILD_SAFETY_EMAIL=PASTE_SAME_OR_ANOTHER_EMAIL
PHOTO_MODERATION=off
```

- If in Step 4.6 you needed the JWT secret, also add `SUPABASE_JWT_SECRET=PASTE_JWT_SECRET`.
- **`PHOTO_MODERATION=off` is for testing only.** It lets every photo through automatically. **You will change this before launch (Step 9.2).**
- `LEGAL_NAME` is the person or company the privacy policy names. It must match the "developer name" on your Google Play listing. Click **Deploy** (or it redeploys by itself).

☐ **Step 5.6. Give it a web address. Settings → Networking → Generate Domain.** You get something like `https://sidequest-api-production.up.railway.app`. Copy it into your

secrets sheet as "Railway API address".

☐ **Step 5.7. Check it is alive.** In your browser open `https://YOUR-RAILWAY-ADDRESS/healthz`. ✅ You see `{"ok":true}`. ❌ If the deploy shows **Failed**: click the deployment → **View logs.** The API prints a list of every missing or wrong variable. Fix them and redeploy. If the **build** fails, send me the last 30 lines of the build log (the Docker build of this server has not been tested by me).

☐ **Step 5.8. Check the legal pages.** Open each of these in your browser:

- `https://YOUR-RAILWAY-ADDRESS/privacy`
- `https://YOUR-RAILWAY-ADDRESS/terms`
- `https://YOUR-RAILWAY-ADDRESS/delete-account`

✅ Each shows a page with your name and email in it. These are the three web links Google Play will ask for. ❌ A "not_found" message means `LEGAL_NAME` / `CONTACT_EMAIL` (or `SUPABASE_ANON_KEY` for the delete page) is missing.

Railway: the worker service

The worker is the part that runs on a timer: reminders, sending check-in alerts, building data exports, deleting accounts after 7 days.

☐ **Step 5.9.** In the same Railway project: **New → GitHub Repo →** `sidequest` again. Rename this service to `worker`. ☐ **Step 5.10.** Its **Settings → Config as code path:** `/apps/api/railway.worker.json`. **Root Directory** `/`. **Do not** generate a domain for it. ☐ **Step 5.11. Variables:** copy the **same variables** as the API (everything in Step 5.5). The worker **refuses to start without** `SUPABASE_SERVICE_ROLE_KEY`. ☐ **Step 5.12.** Open the worker's **logs**. ✅ You see a line `worker started`. Keep the number of replicas at **1**.

PART 6. Moderator page and first sign-in test of the server (20 minutes)

You cannot sign in to the moderator page until your own account exists, so this part continues in Part 8. For now just open `https://YOUR-RAILWAY-ADDRESS/admin` and ✅ confirm you see a page titled "SideQuest moderation" with a sign-in box.

PART 7. Build the app for your own phone and try it (about 1 day, mostly waiting)

7A. Prepare the app settings

☐ **Step 7.1.** In the terminal:

```
cd apps/mobile
```

☐ **Step 7.2.** Log in to Expo and create the project link:

```
eas login
eas init
```

Answer **Yes** to creating a project. ✅ It writes a project id into `app.json`.

☐ **Step 7.3. Tell the cloud build your public settings.** Open `apps/mobile/eas.json` in VS Code. In **all three** places where you see `PASTE_YOUR_…` (in `development`, `preview`, `production`) replace the values:

- `EXPO_PUBLIC_SUPABASE_URL` → your Supabase project URL
- `EXPO_PUBLIC_SUPABASE_ANON_KEY` → your **anon** key (never the service_role key!)
- `EXPO_PUBLIC_API_URL` → your Railway address (starts with `https://`, no slash at the end) Save the file. (These three values are public by design: they ship inside the app.)

☐ **Step 7.4. Create your private `.env` for testing from your computer:**

- Mac/Linux: `cp .env.example .env` Windows: `copy .env.example .env`
- Open `apps/mobile/.env` and fill the same three values. (This file is ignored by Git.)

☐ **Step 7.5. Check the setup:**

```
npx expo-doctor
```

✅ It says all checks passed (a warning or two is fine). Send me anything marked ✖.

7B. Build an installable test app (APK)

☐ **Step 7.6.** Run:

```
eas build -p android --profile preview
```

- Question **"Generate a new Android Keystore?" → Yes**. (This is your app's signing key; Expo keeps it for you.)
- It uploads the project and builds in the cloud. **Wait about 15–25 minutes** (a queue is normal on the free plan).
- ✅ At the end you get a **link and QR code** to the APK. ❌ A build error: send me the last 40 lines.

☐ **Step 7.7. Install it on your phone.** Open the link on your phone (or scan the QR code). Download the file, open it. Android will ask to allow **"install unknown apps"** for your browser: allow it, install, open **SideQuest**.

☐ **Step 7.8. First run.**

1. *Get started* → type your email → **Send code**.
2. ✅ A 6-digit code arrives by email (check spam). If it **never** arrives, go back to Step 4.8–4.9. Also look at *Supabase → Authentication → Logs*.
3. Enter the code → onboarding: birthday (you must be 18+), basics, vibe, energy, interests, **photos**, preferences → **Finish**.
4. ✅ You land in the app with five tabs at the bottom.

> Photos: with `PHOTO_MODERATION=off` your photos are approved instantly, so you will see yourself. Other people's photos are shown through the server, so the Railway API must have `SUPABASE_SERVICE_ROLE_KEY` (it does).

7C. Try everything with two accounts

Real features need **two people**. Use a second phone, an Android emulator, or a friend. Each account needs a different email.

☐ **Step 7.9.** Both accounts finish onboarding in the **same vibe** (for example Friendship). ☐ **Step 7.10. Discovery and likes:** A sees B on Discover → like. B opens the heart icon → sees A → like back → "It's a match". ☐ **Step 7.11. Chat:** send messages; B sees them. Report/block from the chat menu. ☐ **Step 7.12. Invitation:** profile → *Send an invitation* → B accepts → it appears in both calendars. ☐ **Step 7.13. Circles tab:** create a circle, join it with the other account, post a plan, say "I'm going". Try **Social Roulette** (needs 2+ people in the same area text, same activity and time, at least 3 hours ahead; the worker groups them every 5 minutes) and **Double dates** (needs a pair of two people each). ☐ **Step 7.14. Safety area** (Profile → *Safety and privacy*): see your **friend code**; add the other account as a **trusted contact** using their code; schedule a **check-in** for a plan; look at *Your data* (export) and *Blocked people*. ☐ **Step 7.15. A game** inside a chat (the controller icon). ☐ **Step 7.16.** Write down **everything** that looks wrong, ugly, or crashes. Send me the list. This is the most valuable thing you can do. Expect some visual fixes: the screens were rendered only in a fake browser, never on a phone.

7D. If you want live updates while testing (optional)

```
cd apps/mobile
npx expo start --dev-client
```

needs a **development** build (`eas build -p android --profile development`), not the preview one. Skip this unless I ask you to.

PART 8. Make yourself the moderator; turn on push notifications (1 hour)

8A. Become a moderator

☐ **Step 8.1.** In the app, sign in once with **your** email (Part 7 did this). ☐ **Step 8.2.** Supabase → **Authentication → Users** → find your email → copy its **User UID**. ☐ **Step 8.3. SQL Editor → New query**, paste (put your UID in), **Run**:

```
insert into user_roles (user_id, role) values ('PASTE-YOUR-USER-UID',
'admin')
on conflict (user_id) do update set role = 'admin';
```

('admin' can also ban people; 'moderator' cannot.) ☐ **Step 8.4.** Open `https://YOUR-RAILWAY-ADDRESS/admin` → type your email → **Send code** → enter the code → ✅ you see tabs **Reports** / **Appeals** / **Photos**. If it says "Not allowed", check Step 8.3.

8B. Push notifications (recommended before launch, optional for first tests)

Without this everything still works, but nobody gets a notification when the app is closed.

☐ **Step 8.5.** Firebase console → **Add project** (name `sidequest`; Analytics off is fine). ☐ **Step 8.6.** Inside the project: **Add app → Android**. **Package name:** `app.sidequest.android` (exactly). Register. **Download** `google-services.json` and put it in `sidequest/apps/mobile/`. ☐ **Step 8.7.** Open `apps/mobile/app.json`. Inside `"android":`

`{` add this line (with a comma after the line above it):

```
"googleServicesFile": "./google-services.json",
```

Then save it and run `git add . && git commit -m "push config" && git push`. ☐ **Step 8.8.** Firebase → ⚙ **Project settings → Service accounts → Generate new private key** → a `.json` file downloads. Open it, **copy the whole contents** and paste as **one line** into the Railway variable `FCM_SERVICE_ACCOUNT_JSON` on **both** `api` and `worker`. 🔴 Do not commit this file. ☐ **Step 8.9.** Supabase → **Database → Webhooks → Create a new hook** (twice):

Name	Table	Event	URL	Header
`message-created`	`messages`	**Insert**	`https://YOUR-RAILWAY-ADDRESS/v1/hooks/message-created`	`x-webhook-secret` = your `WEBHOOK_SECRET`
`notification-created`	`notifications`	**Insert**	`https://YOUR-RAILWAY-ADDRESS/v1/hooks/notification-created`	`x-webhook-secret` = your `WEBHOOK_SECRET`

(Method **POST**.) ☐ **Step 8.10. Rebuild the app** (`eas build -p android --profile preview`), install it, and in the app turn on notifications when asked. Send yourself a message from the other account with the app closed. ✅ A notification arrives. Notification text never contains names or message text (by design).

PART 9. Before you go public: the readiness checklist (half a day)

Do not move to Google Play until **every** box here is ticked.

☐ **Step 9.1. Email works for strangers.** Custom SMTP is on (Step 4.9) and a code reached an email address that is **not yours**. ☐ **Step 9.2. Turn photo checking ON.** On Railway (**both** `api` and `worker`) **delete** the `PHOTO_MODERATION` variable (or set it to `required`) and redeploy. From now on **new photos stay "pending" until a person approves them** at `/admin → Photos`. **Someone (you) must check that page at least daily**, otherwise new users will have no visible photos. Test it: upload a photo with a test account and approve it in `/admin`. ☐ **Step 9.3. Test account deletion for real** (the server deletes accounts 7 days after the request; here is how to test without waiting):

1. In the app with a **test account**: *Profile → Safety and privacy → Your data → Delete my account*. The app should show a screen saying the account is scheduled for deletion, with a **Keep my account** button. Press it once to prove cancelling works.
2. Request deletion again. In Supabase SQL Editor run (put the test account's UID in):

```
update data_requests set requested_at = now() - interval '8 days'
where kind = 'deletion' and status = 'queued' and user_id = 'PASTE-TEST-ACCOUNT-UID';
```

3. Wait up to 10 minutes. ✅ The test account disappears from *Authentication → Users*, and its photos disappear from Storage. The worker log says `deleted account for request …`.
4. Also test the **web** page: open `/delete-account`, sign in with another test account's email, press **Delete my account**. ☐ **Step 9.4. Test your data export:** *Safety and privacy → Your data → Request my data*. ✅ Within a few minutes it says ready and **Download** opens a file. ☐ **Step 9.5. Read the legal pages out loud.** `/privacy` and `/terms` are **drafts written to match what the app does**. I am not a lawyer. **Have a lawyer (or a qualified legal adviser in your country) read them** and correct them. Check that every statement is true for how you run it. Google compares them to your Data safety form. ☐ **Step 9.6. Check someone reads your support email daily** (`CONTACT_EMAIL`), and that `CHILD_SAFETY_EMAIL` reaches a person who can act quickly. Google requires a child-safety contact who can answer Google about child-safety content. ☐ **Step 9.7. Upgrade Supabase to the paid plan** (backups, no pausing) before real users arrive, and look at *Database → Backups*. ☐ **Step 9.8. Know your moderation duty.** Open `/admin` daily: **Reports** (decide each one), **Appeals** (a person must read them), **Photos**. If you ever find sexual content involving a child, **do not copy or share it**; report it to your national child-safety hotline or, in the US, to the NCMEC CyberTipline (report.cybertip.org), and remove the account. Your published terms promise this. ☐ **Step 9.9. Decide about the wording of one vibe.** The app has vibes named **"Hookups"** ("No strings") and **"Experiment"** ("Explore freely"). Dating apps with casual options do exist on Google Play, but Google restricts sexual content and reviewers use judgment, and **explicit words in the store listing can get you rejected**. For the listing, use only the safe wording in Appendix C. If Google pushes back during review, tell me and I will rename the labels in `packages/shared/src/modes.ts`. ☐ **Step 9.10. App icon and name.** The icon and splash in `apps/mobile/assets/` are placeholders from the build. Replace them with yours if you have them (icon 1024×1024 PNG). Google also needs a separate 512×512 icon for the store listing (Part 11). ☐ **Step 9.11.** Update the app version if you wish: in `apps/mobile/app.json`, `"version": "0.1.0"` is the number users see. (The build number Google needs is increased automatically by Expo.)

PART 10. Build the file you upload to Google Play (about 30 minutes)

☐ **Step 10.1.** Make sure Steps 9.1–9.11 are done and your code changes are pushed to GitHub (`git add . && git commit -m "ready" && git push`). ☐ **Step 10.2.** In `apps/mobile`:

```
eas build -p android --profile production
```

This builds an **AAB** (not an APK). Wait ~20 minutes. ✅ When it finishes, open the build page on expo.dev and **download the `.aab` file**. ☐ **Step 10.3. Back up your signing key (very important).**

```
eas credentials -p android
```

Choose your project → *Keystore* → **download the keystore** and save the file plus its passwords in a safe place (not in the repo). If you lose it you may be unable to update your

app. (When you upload to Google you will use *Play App Signing*, which gives you a safety net: say **yes** to it in Part 11.)

PART 11. Google Play Console: account, app, store listing, forms

Menu names in the Play Console change from time to time. If a name below differs slightly, look for the closest one. Google's help pages are linked at the end of each section.

11A. The developer account

☐ **Step 11.1.** Go to **play.google.com/console/signup**. Sign in with the Google account you want to own the app **for years** (turn on 2-step verification). ☐ **Step 11.2. Choose the account type. This decision matters.**

- **Personal** (you as a person): easiest, but the **12 testers × 14 days** closed-test rule applies (Part 12).
- **Organization** (a registered business): exempt from that rule, but needs a **D-U-N-S number** (free, can take weeks) and business documents. If you are not a registered business, choose **Personal**. ☐ **Step 11.3.** Pay the **one-time US$25**, then complete **identity verification** (government ID, address, phone, and the email shown to users). **This can take several days.** You cannot publish until it is done. Use the *same legal name* you put in `LEGAL_NAME`. Developer verification (Google's newer system that links apps to verified developers) is handled through this same Play Console process for apps published on Google Play, and your Play apps are registered automatically. Read any notices in the Console's inbox and follow them.

11B. Create the app

☐ **Step 11.4.** Play Console → **Create app**. App name **SideQuest**. Default language. **App** (not Game). **Free**. Tick the declarations. *Create app*. ☐ **Step 11.5.** The **Dashboard** now shows a checklist of tasks. You will do each of them below.

11C. "App content" forms (these are the ones people get wrong)

Find them under **Policy and programs → App content** (or the Dashboard checklist).

☐ **Step 11.6. Privacy policy.** Paste `https://YOUR-RAILWAY-ADDRESS/privacy`. (It is a normal web page, not a PDF, and opens without sign-in.) ☐ **Step 11.7. Ads:** *No, my app does not contain ads.* ☐ **Step 11.8. App access** (how Google's reviewers get in). Choose *All or some functionality is restricted* → add instructions like:

```
SideQuest signs in with a one-time code sent by email (no passwords).
Reviewer sign-in: open the app, tap Get started, enter
sidequest.review@gmail.com,
then open https://mail.google.com, sign in with:
  email: sidequest.review@gmail.com    password: <the password you
chose in Step 1.9>
and enter the 6-digit code from the newest email from SideQuest.
The account is already set up for review.
```

Use the reviewer mailbox from Step 1.9. **Before you submit**, sign in once yourself with that address and finish onboarding so the account is ready (and approve its photo in `/admin`).

☐ **Step 11.9. Content rating.** Start the questionnaire. Select the category for **social / dating / communication** apps. Answer honestly: the app **has user-generated content**, users **interact with each other and share their (approximate) location**, there is **no gambling, no ads**, and **no explicit content is provided by the app itself** (content is user-created and moderated). Save. Expect a rating for adults. ☐ **Step 11.10. Target audience and content. Only select 18 and over.** Do not select any child age group. Then follow Google's **age restriction / block minors** instructions for dating apps: Google's rule (policy effective 15 July 2026) says apps whose core function is matchmaking or dating must use Play Console's tools to block minors. Read: `support.google.com/googleplay/android-developer/answer/9867159` and answer every question in that section. ☐ **Step 11.11. Child Safety Standards** (required for social and dating apps). Self-certify that you:

1. **Publish standards that prohibit child sexual abuse and exploitation (CSAE):** your published terms do. Give the URL `https://YOUR-RAILWAY-ADDRESS/terms`.
2. **Provide an in-app way to report concerns:** the app has *Report* on profiles, messages, photos and plans, and *Block*.
3. **Act on CSAM** (remove it and the account) and **report confirmed CSAM** to NCMEC or your regional authority: you must really do this (Step 9.8).
4. **Comply with child-safety laws.**
5. **Provide a child-safety point of contact** (name, email, phone as asked): use a real person who can speak to Google. This is `CHILD_SAFETY_EMAIL`. Do not tick anything you will not actually do. ☐ **Step 11.12. Data safety form.** This is where you tell Google what you collect. Use **Appendix B** as your guide: copy the answers. The answers must **match your privacy policy**. In the section about **data deletion**: answer that users **can request deletion**, give the **web link** `https://YOUR-RAILWAY-ADDRESS/delete-account`, and say **in-app** deletion exists (Profile → Safety and privacy → Your data). Google requires **both** an in-app way and a web link. ☐ **Step 11.13.** Other forms (answer **No/Not applicable** unless it truly applies): *Government apps*, *Financial features*, *Health apps*, *News*, *COVID-19*, *Advertising ID* (the app does not use it).

11D. Store listing (what people see)

☐ **Step 11.14. Main store listing.** Use the safe wording in **Appendix C**. You need:

- App name (30 characters), short description (80), full description (4000).
- **App icon:** 512 × 512 PNG, 32-bit, up to 1 MB.
- **Feature graphic:** 1024 × 500 PNG or JPEG.
- **Phone screenshots:** at least 2 (up to 8), PNG/JPEG. Take them from your own phone during Part 7 (power + volume-down). Use screens that show **no real people's photos or private information**: use test accounts. ☐ **Step 11.15. Category:** *Dating*. **Contact email:** your support email. Website: optional (you can use your Railway address). ☐ **Step 11.16. Countries:** under *Production → Countries/regions*, choose where to release. Start with **your own country** if you are unsure; laws about dating apps and data differ by country. You can add more later.

11E. Signing

☐ **Step 11.17.** When you upload your first file Play will offer **Play App Signing**: say **Yes** (recommended and, for new apps, standard). Google then keeps the real signing key; your Expo keystore becomes the "upload key".

PART 12. The testing path to launch (this is the 4-week part)

Google's tracks, in order: **Internal testing → Closed testing → (apply) → Production**.

12A. Internal test (same day, no review wait, only for you)

☐ **Step 12.1.** Play Console → **Testing → Internal testing → Create new release**. Upload the `.aab` from Step 10.2. Name it `0.1.0`. *Save → Review release → Start rollout.* ☐ **Step 12.2. Testers** tab: create an email list with **your own Gmail** and save. ☐ **Step 12.3.** Copy the **opt-in link**, open it on your phone **logged in with that Gmail**, tap **Become a tester**, then install SideQuest **from Google Play**. ✅ Do the Part 7 checks again on this Play-installed build (sign-in, onboarding, photos, a chat, a plan). If anything differs from the APK you tested earlier, tell me.

12B. Closed test (this is the mandatory 14 days for new personal accounts)

☐ **Step 12.4. Testing → Closed testing → Create track** (call it `Friends`) → create a release with the same (or newer) AAB → *Start rollout.* ☐ **Step 12.5. Recruit at least 12 real people** with Android phones (friends, family, colleagues). Aim for **15–20**, because people drop out. Each needs a **Google account email**. ☐ **Step 12.6.** In the closed track's **Testers** tab, add their emails (a list or a Google Group). Send each person the **opt-in link** with these instructions:

> "Open this link on your Android phone, tap *Become a tester*, then install SideQuest from Google Play. Please keep it installed for the next 14 days and open it a few times. Tell me anything that feels wrong." ☐ **Step 12.7. Keep them opted in for 14 days in a row.** At least **12** must stay opted in **continuously for 14 days**. The Console shows a counter. If it drops below 12, the clock restarts, so keep recruiting extras. (Google says open testing and internal testing do not count; only closed testing does.) ☐ **Step 12.8. Use those 14 days properly.** Fix real problems people report. Upload an **updated build** (increase the version by running a new production build) at least once: Google's production questionnaire asks what you learned and fixed, and "shipped an update because of feedback" is a strong answer. ☐ **Step 12.9. Apply for production.** After the 14 days, the Dashboard shows **Apply for production**. Answer the questions about your testing, your app and its readiness, honestly and specifically (how many testers, what feedback, what you fixed, how moderation works). Google reviews, normally within about a week. If Google rejects, the closed test restarts with the same testers and they tell you why.

12C. Production release

☐ **Step 12.10. Production → Create new release →** *Add from library* (pick your tested AAB) → release notes → **Review → Start rollout to Production**. Google does a final app review (a few hours to a few days; dating apps are checked carefully). ☐ **Step 12.11.** ✅ Your app appears on Google Play. Search for it on your phone. Share the link.

PART 13. After launch, and iPhone later

13A. Running it day to day

- **Every day:** `/admin` → Reports, Appeals, Photos. Answer your support and child-safety emails.
- **Weekly:** Railway logs for errors (`api` and `worker`). Supabase → Reports/Usage for storage and database size.

- **Updating the app:** change code → `git push` → `eas build -p android --profile production` → upload the AAB to a new Play release. Build numbers go up automatically.
- **Changing the database:** only by adding **new** numbered SQL files (`0014_…`). **Never edit a migration you already ran.** Ask me for each new one and paste it into the SQL Editor.
- **Deleting accounts and exports** happen automatically through the worker. Keep the worker running.
- **If Google or a user asks to remove content or an account**, do it in `/admin` and keep a note.
- **Money:** the app is free forever by design. Your costs are hosting (Supabase, Railway).

13B. iPhone / Apple App Store (not set up, not tested)

The app was written to run on iPhone later, but **nothing for iOS has been configured or tested**. When you are ready, ask me to prepare it. What it will need from you: an **Apple Developer Program** membership (about US$99/year), an app icon set, a **demo account** for Apple's reviewers, Apple's **age rating (18+)**, privacy "nutrition labels", and a build via `eas build -p ios` followed by **TestFlight** testing. Apple's rules for apps with user content are similar to Google's (report, block, in-app account deletion, contact information), and the app already has those features.

APPENDIX A. Which setting goes where

Variable	Railway `api`	Railway `worker`	App (`eas.json`)	Notes
`DATABASE_URL`	✅	✅	❌	🔴 session pooler string
`SUPABASE_URL`	✅	✅	as `EXPO_PUBLIC_SUPABASE_URL`	
`SUPABASE_ANON_KEY`	✅	✅	as `EXPO_PUBLIC_SUPABASE_ANON_KEY`	public by design
`SUPABASE_SERVICE_ROLE_KEY`	✅	✅	❌ **never**	🔴
`SUPABASE_JWT_SECRET`	only if Step 4.6 needed it	same	❌	
`WEBHOOK_SECRET`	✅	✅	❌	🔴 20+ characters
`FCM_SERVICE_ACCOUNT_JSON`	✅	✅	❌	🔴 one line
`LEGAL_NAME`, `CONTACT_EMAIL`, `CHILD_SAFETY_EMAIL`	✅	optional	❌	switches the legal pages on
`PHOTO_MODERATION`	`off` while testing, **removed** for launch	same	❌	Step 9.2
(Railway address)			as `EXPO_PUBLIC_API_URL`	https, no trailing slash

APPENDIX B. Data safety form: suggested answers

These match what the app actually does. **Re-read Google's current definitions before you submit, and make sure your privacy policy says the same.** If you add anything later (analytics, crash reporting, ads), update both.

Data type (Google's category)	Collected?	Shared with third parties?	Required or optional	Why
Email address (Personal info)	Yes	No*	Required	Account management, sign-in
Name (display name)	Yes	No*	Required	App functionality (shown to other members you choose)
User IDs	Yes	No*	Required	Account management
Other personal info: date of birth, gender	Yes	No*	Required	Age check (18+), app functionality
Sexual orientation (if your profile asks who someone wants to meet, treat it as this)	Yes	No*	Optional	App functionality (matching preferences)
Approximate location	Yes	No*	Optional	Finding people and plans nearby
Photos	Yes	No*	Optional (needed for a good profile)	Profile and chat photos
Messages (in-app messages)	Yes	No*	Optional	Chat
Other user-generated content (profile text, plans, game answers)	Yes	No*	Optional	App functionality
App activity (likes, visits, invitations)	Yes	No*	Required	App functionality, safety
Device or other IDs (push notification token)	Yes	No*	Optional	Notifications
Contacts, calendar from phone, health, financial, audio, video, web browsing, crash logs	**No**			

* Google does not count data processed **for you by service providers** (Supabase, Railway, Firebase Cloud Messaging) as "sharing". Your policy names them. Check Google's current definition of "sharing" before you answer.

Security section: **Data is encrypted in transit: Yes. Users can request data deletion: Yes** (in-app and at the web link). Independent security review: No.

APPENDIX C. Store listing text (safe wording)

App name: `SideQuest: Meet & Do Things`

Short description (80): `Meet people and join real plans near you. Free for everyone, always.`

Full description (draft):

```
SideQuest is a free app for adults (18+) who want to meet people and
actually do things together.

CHOOSE YOUR VIBE
Friendship, dating, adventures and more. You pick the vibe you are in,
and you only appear to people looking for the same thing.

DO THINGS TOGETHER
• Circles: join groups around your interests and plan meet-ups
• Open calls and activities: invite people to a walk, a game night or a
coffee
• Social Roulette: tell us what you fancy doing and we seat you with a
small group in a public place
• Double dates: bring a friend and meet another pair
• Quick games to break the ice in chat

SAFETY BUILT IN
• Report and block anyone, any time
• Photos are checked before others see them
• Trusted contacts and check-ins after you meet someone
• A wingperson can help with your profile, but can never read your
chats
• Delete your account whenever you like, in the app or on the web

FREE, FOREVER
No paywalls. Likes and visitors are never blurred or hidden.

You must be 18 or older to use SideQuest. By using it you agree to our
Terms and Community Guidelines and Privacy Policy.
```

Do not use in the listing: "hookup", "sex", "no strings", "explore freely", "casual encounters", or anything sexual. Do not use another company's name or logo.

APPENDIX D. Common problems and fixes

What you see	Most likely cause	Fix
`node` / `pnpm` / `git` "not recognized"	Terminal opened before install	Close it, open a new one; reinstall if needed
`pnpm test` fails in the mobile block on a slow computer	Machine under heavy load	Close other programs and run `pnpm test` again; if it still fails, send me the lines
Railway deploy **Failed** at start	A variable is missing or wrong	Open the logs; the API lists exactly which; fix and redeploy
Railway build fails	Docker build untested by me	Send me the last 30 lines of the build log
`/privacy` says "not_found"	`LEGAL_NAME` or `CONTACT_EMAIL` not set	Add them on the `api` service
App: "Missing EXPO_PUBLIC_SUPABASE_URL…"	`eas.json` still has `PASTE_YOUR_…`	Step 7.3, rebuild
Sign-in code never arrives	Email templates or SMTP not set	Steps 4.8–4.9; check Supabase Auth logs and spam
Everything says "Please sign in again" / 401	Project signs logins with a legacy secret	Step 4.6: add `SUPABASE_JWT_SECRET` to `api` and `worker`
No photos visible	`PHOTO_MODERATION` on and nobody approved, or service key missing	Approve in `/admin → Photos`, or check `SUPABASE_SERVICE_ROLE_KEY` on `api`
`/admin` says "Not allowed"	You are not in `user_roles`	Step 8.3
No push notifications	Firebase/webhooks/rebuild missing	Steps 8.5–8.10; check Railway logs
Roulette never seats anyone	Not enough people with the **same** area text, activity and time	Use identical words; the worker runs every 5 minutes
Google rejects the upload: "target API level"	Old build	Production builds target API 36; rebuild with the latest code

Google says the 12-tester rule is not met	Fewer than 12 opted in for 14 days in a row	Add more testers; keep them opted in

APPENDIX E. What I could and could not check for you

Checked (in my sandbox): the unzip → `pnpm install` → `pnpm test` → `pnpm typecheck` → `pnpm check:sql` path from a fresh copy of this zip; the server builds; the generated Android project targets API level 36, uses the package `app.sidequest.android`, and requests only approximate location and notifications (plus Expo's default overlay and vibration); all 13 SQL files apply on a normal Postgres 16 and the safety self-test passes; the privacy, terms and delete pages work and are safe against injected text; Google's current Play rules were read on Google's own pages on 7 November 2026.

Not checked (only you can): anything on a real phone; Supabase, Railway, Firebase, Expo and Google Play as real services; whether your legal texts are right for your country (get a lawyer); whether Google approves a dating app; delivery of sign-in emails and push notifications; the Docker build of the server on Railway.

When you hit the first problem, send it to me (Part 0.4) and we will fix it together.